# BUSINESS ANALYST INTERN TECHNICAL ASSESSMENT

## Question Set: B

Congratulations on clearing the aptitude round. This assessment has four sections, all built on a single CSV. Section 2 is a substantial CASE STUDY read its briefing carefully. The mini-project (Section 4) is the only open-ended deliverable; everything else has specific, verifiable answers.

*Suggested time: 4 hours, submit a single ZIP. Read all footnotes before starting they document real data quirks that will sink surface-level analysis.*

## The Dataset

A single CSV (hotel bookings.csv) of 12,000 booking records from a hotel-booking platform. Each row is one booking; customer and property attributes are denormalized into the row. 28 columns:

- **Booking id, customer id, property id** — Primary key + foreign keys
- **Customer name, customer segment, customer signup date, customer home city, customer loyalty tier** — Customer attributes (denormalized into every row)
- **Property name, property city, property star rating, property type, property total rooms** — Property attributes (denormalized into every row)
- **Booking date, check-in date, checkout date** — Key dates
- **Room type, num rooms, nights** — Room details
- **Booking channel** — Direct Website / OTA / Travel Agent / Corporate Portal
- **adr, discount amount, coupon code, total amount, payment method** — Pricing & payment (total amount = adr * nights * num rooms – discount amount)
- **Booking status** — Completed / Cancelled / No-Show
- **Review rating, review date** — Review info (null if no review left)

### Important Footnotes read before starting

**Footnote 1 Invalid stays:** Some rows have checkout date on or before check-in date. Treat as data errors.

**Footnote 2 Booking before account:** Some rows have booking date earlier than customer signup date a temporal-integrity issue.

**Footnote 3 Zero rooms:** Some rows have num rooms = 0 (carry total amount = 0). These are erroneous.

**Footnote 4 Property names repeat across cities:** A handful of property name values appear in multiple cities with DIFFERENT property ids. NEVER group on property name always use property id.

**Footnote 5 Reviews on Cancelled bookings:** A small number of Cancelled bookings carry a review rating, which is impossible. Flag these.

**Footnote 6 Review scale mismatch:** review rating is on 1-5 for Individual and Group customers, but on 1-10 for corporate customers. Without normalization, any cross-segment rating average is wrong.

**Footnote 7 customer loyalty tier 'None' parsing trap:** The tier column has 4 valid values including the literal string 'None' (meaning 'no tier'). Default CSV readers (pandas read csv, etc.) will silently convert 'None' to a missing value, making ~46% of your tier data disappear. Use keep default Na=False (pandas) or the equivalent in your tool.

**Footnote 8 Realized vs booked revenue:** Only booking status = 'Completed' counts as realized revenue. Cancelled and No-Show rows still carry a total amount (the would-have-been-charged amount); summing all rows overstates revenue by ~30%.

## Section 1 Data Quality Checks (quick)

**B1.** How many distinct property name values appear in more than one property city (different property ids)? List them.

**B2.** The coupon code column has TWO encodings for 'no coupon'. Name both and report the % of bookings using a real coupon.

**B3.** How many review rating values exist on Cancelled bookings? Why is this impossible — propose one rule for handling them.

## Section 2 Case Study: The Discount Erosion Problem

> **Briefing:** The CFO has noticed that discount spend as a share of gross revenue has been creeping up. Marketing argues that coupons are essential to drive volume on competitive channels. The CFO suspects we are bleeding margin without proportional behavioural lift, especially on the OTA channel. The CFO wants to know: (a) where exactly is the discount intensity concentrated, (b) are those discounts actually moving the dial on cancellation, repeat behaviour, or basket size — or are they pure margin leakage, and (c) what's the recoverable margin if we pulled back, with the risk quantified.

> **What we expect:** You must include at least TWO labelled visualizations: one comparing discount intensity across channels (Q1) and one comparing coupon vs. no-coupon behaviour on the focus channel (Q3). Every conclusion must be backed by specific numbers — 'coupons aren't working' is not an answer; 'cancel rate is x% with coupons vs y% without, repeat rate x% vs y%, per-room amount ₹x vs ₹y' is.

**B1 Discount Landscape (visualization required).**

Compute discount intensity (sum of discount amount ÷ gross revenue, where gross = total amount + discount amount, both summed over Completed bookings) for each booking channel. Produce a labelled chart. State the platform-wide intensity AND name the channel that is most above the platform average, with the gap in percentage points.

**B2 Is the Discount Channel Different in Other Ways?**

For the highest-intensity channel, compare its CUSTOMER MIX against the platform: shares by customer segment, by customer loyalty tier (remember Footnote 7), and by source-city pattern. Is this channel discount-heavy because of WHO uses it (the customers are different), or is the channel itself running heavier discounts? Show the mix tables.

**B3 Are the Coupons Actually Doing Anything? (Visualization required).**

Within the highest-intensity channel ONLY, compare coupon vs no-coupon bookings on three behavioural metrics: (i) cancellation rate, (ii) per-room amount (total amount ÷ num rooms, excluding num rooms = 0). Show the side-by-side numbers. State plainly: do coupons attract better behaviour, identical behaviour, or worse behaviour?

**B4 Recommendation + One Leading Indicator.**

Propose a SPECIFIC pullback strategy (e.g., 'remove coupons for customers with previous bookings on the channel', 'cap discount per booking at X%', 'discontinue coupon code Y'). State the expected margin recovery in ₹. Then name ONE leading indicator from this dataset (a specific column or metric) that the team should monitor in the first 60 days to confirm the pullback is not hurting volume — and give the threshold at which you would abort the rollback.

## Section 3 SQL Challenge

*Tests SQL fluency AND schema understanding. The dataset is denormalized; your first task is to design a normalized schema. The queries are written against the schema YOU design.*

**Schema Design Task (worth as much as the queries):**

Design a normalized schema by writing CREATE TABLE statements for at least three tables (customers, properties, bookings, optionally reviews). For every column specify:

- SQL data type (INTEGER, VARCHAR(n), DECIMAL(p,s), DATE, etc.) think precision for money and length for free text;
- primary key, foreign key, or neither plus NOT NULL where appropriate;
- ONE non-trivial CHECK or UNIQUE constraint addressing one of the footnotes (e.g., CHECK(checkout date > check-in date) for Footnote 1; UNIQUE(property name, property city) for Footnote 5);
- ONE column to index for speeding up your queries below, with a 1-2 line justification.

**SQL Queries (against your normalized schema, or against the flat CSV loaded as one table state which):**

**B-Q1.**

For each property type, find the room type with the most Completed bookings using DENSE RANK () partitioned by property type. Return property type, room type, booking count. In 1-2 lines, explain why DENSE RANK fits here vs. RANK or ROW NUMBER.

**B-Q2.**

Compute monthly realized revenue for 2024 with a running cumulative total using SUM () OVER (ORDER BY month). Output: month, monthly revenue, cumulative revenue. Report the full-year cumulative value.

## Section 4 Mini-Project (build something + integrate a free API)

Build a small WORKING tool or notebook that enriches the dataset using a live, free, no-key public API and produces one concrete, quantified insight. You MAY (and should) use AI assistants we expect it. But the deliverable must be a working artifact YOU wired together, debugged, and can explain. We are assessing how well you USE AI to BUILD, not whether AI can answer a prompt.

### Set B Holiday-Proximity Demand Tagger

API: Nager Date Public Holidays API free, no key. Endpoint: https://date.nager.at/api/v3/PublicHolidays/2024/IN.

What to build: Pull the 2024 India public-holiday calendar. Tag every booking by whether its check-in date falls within ±2 days of a public holiday (a 'long-weekend' booking). Compare booking value, length of stay, and cancellation rate for holiday-adjacent vs ordinary bookings. Quantify the lift (or lack of it).

**Every mini-project must include ALL of the following:**

1. A real call to the assigned free API, with error handling timeouts or missing data should NOT crash the run. Show this in the code.
2. A correct merge of the API data back onto the bookings (watch join keys, dates, city country/coords mappings).
3. ONE non-obvious, quantified insight the dataset ALONE could not produce a number or chart, stated in 3-4 sentences.
4. A 6-line README: what it does, how to run it, one design decision, one limitation.
5. A specific note (≤ 8 lines) on where you used AI and what you had to FIX in the AI's output vague notes lose marks; specific notes win.

## Deliverables & How We Grade

Submit a single ZIP containing:

1. answers.pdf (or .docx): Sections 1-3 your CREATE TABLE statements + index justification, your SQL queries pasted as text, and the case-study answers with embedded visualizations.
2. code/ folder: your analysis notebook/script + SQL files.
3. project/ folder: working mini-project code + README + the insight + AI-usage note.